

Decoder Reduction Approximation Scheme for Booth Multipliers

Muhammad Hamis Haider , Hao Zhang , *Member, IEEE*, and Seok-Bum Ko , *Senior Member, IEEE*

Abstract—Existing approximate Booth multipliers fail to keep up with modern approximate multipliers such as truncation-based approximate logarithmic multipliers. This paper introduces a new approximation scheme for Booth multipliers that can operate with negligible error rates using only $N/4$ Booth decoders, instead of the traditional $N/2$ Booth decoders. The proposed 16-bit BD16.4 approximate Booth multiplier reduces the Normalized Mean Error Deviation (NMED) by 96.5% and the Power-Area-Product (PAP) by 69.6%, when compared to a state-of-the-art approximate logarithmic multiplier. Additionally, the proposed BD16.4 approximate multiplier reduces the NMED by 94.4% and PAP by 74.8%, when compared to a state-of-the-art higher-radix approximate Booth multiplier. The proposed 8-bit approximate Booth multipliers reduce the NMED by up to 74% and PAP by up to 5% when compared to the existing state-of-the-art approximate logarithmic multipliers. We validated the results derived in this paper through a neural network inference experiment, where the proposed approximate multipliers showed a negligible drop in inference accuracy compared to the exact Booth multipliers and the state-of-the-art approximate logarithmic multipliers (ALM). The proposed approximate multipliers achieved a Power-Delay-Product reduction of 63% (vs. exact) and 21.22% (vs. ALM) in 16-bit experiments and a reduction of 67% (vs. exact) and 8.75% (vs. ALM) in 8-bit experiments.

Index Terms—Booth multipliers, approximate computing, convolutional neural networks, logarithmic multipliers, leading one detection.

I. INTRODUCTION

APPROXIMATE computing forgoes output accuracy for an efficient design of a computing unit. The hardware designers look for sensible potential shortcuts and eliminations in the original circuit to achieve a less resource-hungry implementation, albeit with output errors. Approximate multipliers (AM) are very popular in recent academic publications for error-tolerant applications such as image processing, object detection, edge detection, and deep neural network (DNN) inference [1], [2], [3], [4].

Manuscript received 11 May 2023; revised 19 October 2023; accepted 10 December 2023. Date of publication 14 December 2023; date of current version 12 February 2024. Recommended for acceptance by K. Gaj. (*Corresponding author: Seokbum Ko.*)

Muhammad Hamis Haider and Seok-Bum Ko are with the Department of Electrical and Computer Engineering, University of Saskatchewan, Saskatoon, SK, S7N5A2, Canada (e-mail: hamis.haider@usask.ca; seokbum.ko@usask.ca).

Hao Zhang is with the Faculty of Information Science and Engineering, Ocean University of China, Qingdao 266100, China (e-mail: hao.zhang@ouc.edu.cn).

Digital Object Identifier 10.1109/TC.2023.3343093

The variation and magnitude of the deviation of the AM output depend on the applied approximation schemes. Truncation of the output bits is the simplest approximation technique that can be applied to any multiplier to achieve a higher resource efficiency; however, this approximation performs poorly for small input values due to a very high relative error since the error introduced under truncation remains static across all the possible inputs [5].

Booth multipliers and logarithmic multipliers are two popular multiplier architectures for approximation in the literature [6]. Both multiplier architectures convert the input binary numbers into an alternate representation, Booth encodes and logarithmic scale respectively, before they compute the final product. The architecture-specific approximation techniques possible in these multipliers are much more sophisticated than the simple truncation and have a less consequential error pattern in the output. Approximate Booth multipliers employ mixed-radix, high-radix, partial product truncation, and approximate partial product generators to implement approximation in the multiplier circuit. In contrast, approximate logarithmic multipliers employ dynamic truncation, operand-trimming, and hybrid architecture to implement approximation in the multiplier circuit. Generally, approximate logarithmic multipliers have been targeted solely for Convolutional Neural Network (CNN) inference, whereas approximate Booth multipliers (ABM) work well across most error-tolerant applications. Consequently, approximate logarithmic multipliers have a higher resource efficiency and error rate than the approximate Booth multipliers [7].

Dynamic truncation [8] is a middle-ground between complicated architectural approximation techniques and the simpler truncation technique. In its common implementation [9], a leading one-detector (LOD) is used to identify the most impactful bit-position from the provided input binary numbers. A certain number of bits after the leading one are retained and considered for the remainder of the multiplication process. The LOD circuit is employed in a multiplier circuit to identify the bit-position with the highest impact in the input [10], [11]. As we move towards the right in any binary number, the weightage of the bit-position decreases by a power of 2. It makes sense, that should the first 1 occur in a binary number at bit-position 16 (with a weightage of $2^{16} = 65536$), we can safely ignore the bit-positions of 8 or less, with a combined weightage of ($2^9 - 1 = 511$). We can do this because the overall percentage of contribution of the right-most 9 bits compared to the first 1 in the bit-position 16 is 0.78%.

Modern logarithmic multipliers take advantage of this fact and there are several state-of-the-art approximate logarithmic multipliers proposed in the literature that exploit this feature of the LOD-based truncation schemes. Most notable among these are the multipliers presented in [12], [13], [14], [15]. The authors in these papers have successfully shown that the approximate logarithmic multipliers are capable of extremely efficient operation and that they are best suited for error-tolerant applications in the field of Deep Learning (DL).

The LOD-based approximation techniques are not common in approximate Booth multipliers due to a mismatch between the LOD method and the internal architecture of the Booth multiplier. The Booth multiplier works on the principle of encoding the input numbers in a convenient format known as Booth encodes and then generating decodes which are accumulated together to reach the final product. The encodes are generated using overlapping slices of the initial input binary number, and a Booth multiplier is extremely sensitive to any truncation or modifications in the bit-positions of the input binary numbers [16]. Instead, the truncation usually occurs on the side of the output port, and the approximation schemes are targeted at the Booth encoder blocks [17] or the Booth decoder blocks [16], [18].

In this paper, we propose a novel approximation scheme that aims to exploit the potential of the existing Booth encoding scheme to achieve a circuit simplification for approximate computing. We achieve a similar effect as LOD-based truncation approximation schemes do, wherein, we reduce the total number of computational elements and reduce the height of the adder trees to improve the resource efficiency of the multiplier. The proposed approximation scheme focuses on reducing the need for $N/2$ decoders in a traditional Booth multiplier. We demonstrate that through the proposed approximation scheme, a Booth multiplier can operate in approximate computing mode with $N/4$ decoders (50% reduction) with a negligible error rate. The contributions of our work are as follows:

- We propose a novel approximation scheme called the Decoder Reduction Approximation (DRA) scheme that allows the reduction of Booth decoders from D_{max} ($N/2$) to any smaller number.
- We propose multiple approximate Booth multipliers using the proposed DRA scheme, with a special focus on the proposed designs with $D_{max}/2$ decoders in them.
- The proposed approximate multipliers outperform the existing state-of-the-art approximate logarithmic multipliers which have been shown in the literature to be more efficient than the existing approximate Booth multipliers.,
- The proposed DRA scheme and the proposed approximate multipliers are implemented for the Neural Network inference task. They outperform the existing state-of-the-art approximate multipliers in terms of accuracy compared to resource efficiency.

The rest of the paper is organized as follows: Section II discusses the proposed DRA scheme for error-tolerant Booth multipliers, Section III describes the proposed designs component-wise in detail, Section IV explains the

experimentation setup and results, and finally in Section V we conclude our findings.

II. THE PROPOSED DECODER REDUCTION APPROXIMATION (DRA) SCHEME

In this paper, we propose a novel approximation scheme called decoder reduction approximation (DRA). The proposed DRA scheme is optimized for the Booth algorithm and all existing Booth multiplier architectures in the literature are compatible with it. Under the proposed scheme, a reduced number of decoders are implemented and the incoming Booth encoded signals are filtered out depending on their contribution percentage to the final product.

Unlike the LOD-based truncation schemes, where a T number of bits after the leading one bit are chosen, in the proposed DRA scheme, the bits are not restricted to be consecutive. Given a Booth multiplier with N -bit inputs, in the proposed DRA scheme, we pick the first W non-zero decodes from the D_{max} incoming Booth decodes. W is the specified number of chosen decodes, always lower than D_{max} and D_{max} is the maximum number of Booth decodes possible, it is always equal to $N/2$. Since the proposed DRA scheme is not operating directly on the input bits, it allows for overlapped and disjointed bits to be selected. The proposed DRA scheme has a higher degree of freedom compared to the LOD schemes available for modified Booth multipliers. For example, Given two inputs: $A = (0100011)_2$ and an arbitrary 8-bit binary number B , and LOD truncation set to 4-bits after the leading-one, and W set to 2 for our proposed DRA scheme (equivalent to 4-bit LOD due to Booth encode overlap). In a truncated-based approximation scheme for Booth multipliers, B input would be used to generate 4 Booth encodes and the truncated A input would generate 5-bit decodes. Whereas, in our proposed DRA scheme, a lower-amount of Booth encodes are generated using A input itself. Below we compare the higher degree of flexibility available in the proposed DRA scheme for approximate Booth multipliers:

$$\begin{aligned} LOD_out &\leftarrow (10000)_2 \\ DRA_out &\leftarrow E_0 : BoothEncodeFrom(010) \\ &\quad \leftarrow E_1 : BoothEncodeFrom(001) \end{aligned}$$

So, if we underline the active bits for each approximation scheme, we get:

- LOD: (0 10000 11)₂, which is less flexible and completely ignores the right-most portion.
- Proposed DRA: (010 0 001 1)₂, which is more flexible and allows for lower error deviations.

Additionally, the proposed DRA scheme also implements priority selection inspired by a priority encoder circuit to reduce the circuit complexity for the filtration logic in the multiplier.

The highly flexible design of the proposed DRA scheme allows the multiplier to maintain a low error rate even with a half or quarter number of decoders compared to an exact Booth multiplier. The low error rate allows the DRA scheme to be compatible with the traditional truncation approximation which further increases the resource efficiency of the proposed multipliers. The filtration logic for the multipliers depends

TABLE I
THE INCREASE IN COMPLEXITY AS THE
DIFFERENCE BETWEEN N AND W INCREASES.
WE HAVE HIGHLIGHTED THE UNFEASIBLE
VARIATIONS OF N AND W DUE TO HIGHER
COMPLEXITY

N ¹	N/2	W ³	$N/2 C_W$	Labels ²
8	4	1	4	BD8.1
	4	2	6	BD8.2
	4	3	4	BD8.3
16	8	1	8	BD16.1
	8	2	28	BD16.2
	8	3	56	BD16.3
	8	4	70	BD16.4
	8	5	56	BD16.5
	8	6	28	BD16.6
	8	7	8	BD16.7

¹ The bit-width of the multiplier.

² The multipliers that are discussed in detail are referred to using these labels in the rest of this paper.

³ The number of decoders used under the proposed DRA scheme (See Section II).

on the bit-width, and the user-specified number of decoders, referred to as W in this paper. The filtration logic under the proposed DRA scheme can be calculated using the maximum number of combinations ($n C_r$). Due to this relationship, as the difference between the maximum possible decoders (D_{max}) and the chosen number of the decoders (W) increases, the circuit becomes increasingly complex. Table I lists the total number of combinations for each possible value of W that can be used. For an 8-bit implementation, there aren't any serious implementation complications; however, in the 16-bit multipliers, the scheme requires further optimizations before it can be implemented properly in the hardware.

The proposed DRA scheme follows the workflow as outlined in Algorithm 1. For the benefit of the readers, we point out that the major points of focus in our hardware implementations are the trade-offs made in the critical path of the circuit and the reduction in the resource consumption of the proposed multiplier compared to an exact 8-bit Booth multiplier. We highlight that the proposed scheme adds a step in the Booth multiplier workflow at line number 4 in Algorithm 1. In this scheme, we show that the DRA scheme makes up for these additions in the critical path by reducing the complexity of the adder trees required to accumulate the partial products.

III. THE PROPOSED APPROXIMATE BOOTH MULTIPLIERS

Our proposed designs are named as $BD\ N.W$, where BD denotes a unique prefix to differentiate the proposed designs compared to the designs from the literature in the reported experimental results, N stands for the bit-width of the proposed approximate Booth multiplier (8 or 16), and W stands for the specified number of decoders under our proposed DRA scheme. W is defined by us, and it should always be less than $N/2$. In Table I, we have provided a list of labels and the total number of combinations possible.

Algorithm 1: The workflow of the proposed DRA scheme.

Input: [N-1:0] inA, [N-1:0] inB
Output: [2*N-1:0] OutZ
 /* Maximum number of decoders possible given N-bit inputs */
 1 $D_{max} \leftarrow N/2$;
 /* W is given by the user, and should be less than N/2 */
 2 $W \leftarrow \text{Given}$
 /* Get the Booth encodes */
 3 $encs[D_{max} - 1:0] \leftarrow BE(inA)$;
 /* Filter W out of D_{max} Booth encodes */
 /* (See Table II) */
 4 $flt_encs[W - 1:0] \leftarrow Fil(encs[D_{max} - 1:0])$;
 /* Get W decodes using the W filtered encodes */
 /* BD() returns the Booth decodes for all the input encodes */
 5 $dra_D[W - 1:0] \leftarrow BD(flt_encs[W - 1:0], inB)$;
 /* Finally add the decodes using adder trees (AT function). */
 6 $OutZ \leftarrow AT(dra_D[W - 1:0])$;

TABLE II
THE ENCODE FILTRATION LOGIC FOR AN 8-BIT BOOTH
MULTIPLIER USING THE PROPOSED DRA SCHEME

Z_3^2	Z_2	Z_1	Z_0	$W^1 = 1$	$W = 2$	$W = 3$
0	0	0	0	E_3^*	E_3, E_2	E_3, E_2, E_1
0	0	0	1	E_3	E_3, E_2	E_3, E_2, E_1
0	0	1	0	E_3	E_3, E_2	E_3, E_2, E_0
0	0	1	1	E_3	E_3, E_2	E_3, E_2, E_0
0	1	0	0	E_3	E_3, E_1	E_3, E_1, E_0
0	1	0	1	E_3	E_3, E_1	E_3, E_1, E_0
0	1	1	0	E_3	E_3, E_0	E_3, E_1, E_0
0	1	1	1	E_3	E_3, E_2	E_3, E_1, E_0
1	0	0	0	E_2	E_2, E_1	E_2, E_1, E_0
1	0	0	1	E_2	E_2, E_1	E_2, E_1, E_0
1	0	1	0	E_2	E_2, E_0	E_2, E_1, E_0
1	0	1	1	E_2	E_2, E_1	E_2, E_1, E_0
1	1	0	0	E_1	E_1, E_0	E_2, E_1, E_0
1	1	0	1	E_1	E_1, E_0	E_2, E_1, E_0
1	1	1	0	E_0	E_1, E_0	E_2, E_1, E_0
1	1	1	1	E_0	E_1, E_0	E_2, E_1, E_0

¹ $W \rightarrow$ The number of decoders under the proposed DRA scheme.

² $Z_y \rightarrow 1$ if the corresponding D_y (Yth Booth decode) is expected to be zero, and 0 otherwise.

* $E_x \rightarrow$ Xth Booth encode in an 8-bit Booth multiplier.

A. Proposed 8-Bit ABMs Using the Proposed DRA Scheme

We employed unique filtration logic in each of the 8-bit versions of the proposed multipliers as shown in Table II.

There are several internal components that are shared by each of the proposed 8-bit approximate multipliers. The encode select unit takes in D_{max} encodes and then outputs W encodes.

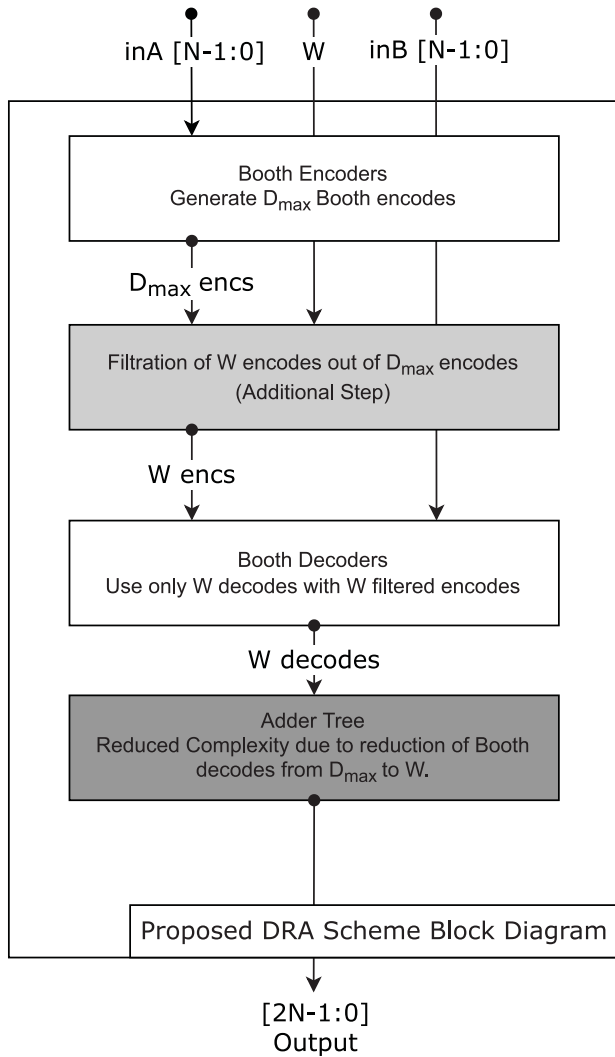


Fig. 1. Block diagram of the proposed DRA scheme for an N-bit approximate Booth multiplier. Where D_{max} is equal to $N/2$.

This is shown using z_a (zero signal), n_b (negation signal), and t_c (two/shift signal) for each of the output encodes from the encode select unit, as shown in Fig. 2. The numbers replacing the a , b , and c in the subscript of the signal show the row position of the partial product. The truncation unit can be included in any design, and it simply truncates the input applied at port i_b to L-bits. We add the truncation unit in the design of the overall error rate of the proposed multiplier can tolerate further approximations. So, L is set to N if the truncation is not operational in the hardware of the proposed multipliers. The Booth sign extension unit applies the compression sign extension required for each of the partial products as per the regular partial product accumulation initially proposed in [19]. After the appropriate sign extension is applied then the decode is shifted an appropriate amount as dictated by the pp_sel signal. For example, if E_2, E_1 are chosen in BD8.2, then this unit applies a logical left shift of 2-bit positions to E_2 . After the addition stage, a final shift module is required to compensate for the truncation (if applied) and to apply a final shift for the

right-most encode chosen. If E_3, E_2 are chosen in BD8.2, then a left logical shift of 4 is applied, and if E_3, E_2, E_1 are chosen in BD8.3, then a final left logical shift of 2 is applied.

The general figure of the proposed DRA multiplier shows the internal modules in groups shown by a shade either dark or light. The light gray shade shows the novel modules for encode selection and then shift compensation, whereas the very dark gray shade shows the decode generation and the compressed sign extension blocks. The truncation unit is in its group with a medium dark gray shade. The purpose of these blocks is to highlight the close relationship between the modules in the same group.

1) *BD8.1—Proposed Multiplier With $W = 1$* : The proposed BD8.1 multiplier works using a MUX selection logic that follows a priority scheme, where if a Booth decodes with a higher weightage is non-zero then it is preferred on the rest of the Booth decodes as shown in Table II. This is shown in Table II, wherein as long as Z_3 is non-zero the multiplier will always pick encode E_3 to generate the single Booth decode and so on. The proposed BD8.1 can run at a fraction of the resource consumption compared to exact multipliers due to the calculation of a single Booth decode. We include this variation of the proposed ABMs to ensure a complete and thorough discussion of this new approximation scheme.

In BD8.1, a single Booth decode is chosen from the 4 (D_{max}) available. The chosen decode is then sign-extended using the compressed sign extension method and then a shift is applied. There is no ripple carry adder in this proposed multiplier. Essentially, this multiplier operates very similar to a barrel shift multiplier and the power-of-2 only Booth multiplier [20], except that it can accept the full range of the signed 8-bit input instead of just pre-encoded values. Since, there is a contrast in scope, a comparison between the two is not possible.

2) *BD8.2—Proposed Multiplier With $W = 2$* : The encode select block in the proposed BD8.2 multiplier works the same as the proposed BD8.1 multiplier, except that here it has to choose 2 encodes out of 4 (D_{max}). The choice of an additional encode causes the selection network to become relatively complex since the choice of the second encode depends on the first encode. This behavior can be observed in Table II. The first encode replicates the $W = 1$ column, however, to choose the second encode, we first need to figure out which encode was chosen in the first place.

Since W is even and half of D_{max} in this variation, the overall design is still simpler compared to BD8.3. The overall accumulation process in this variation also remains regular. This variation provides the best amount of trade-off between the error rate and the gains in resource efficiency. In our experiments, the proposed BD8.2 performed extremely well, therefore, we included the truncation unit in this design. We tested the proposed BD8.2 with L set to 8, 6, or 4. The truncation combined with the 50% reduction in the number of decoders makes the proposed BD8.2 multiplier a good choice for resource-constrained hardware in error-tolerant applications such as DL or DSP.

3) *BD8.3—Proposed Multiplier With $W = 3$* : Since the encode select unit chooses 3 encodes out of 4 (D_{max}) the error rate of the proposed BD8.3 multiplier is the lowest compared

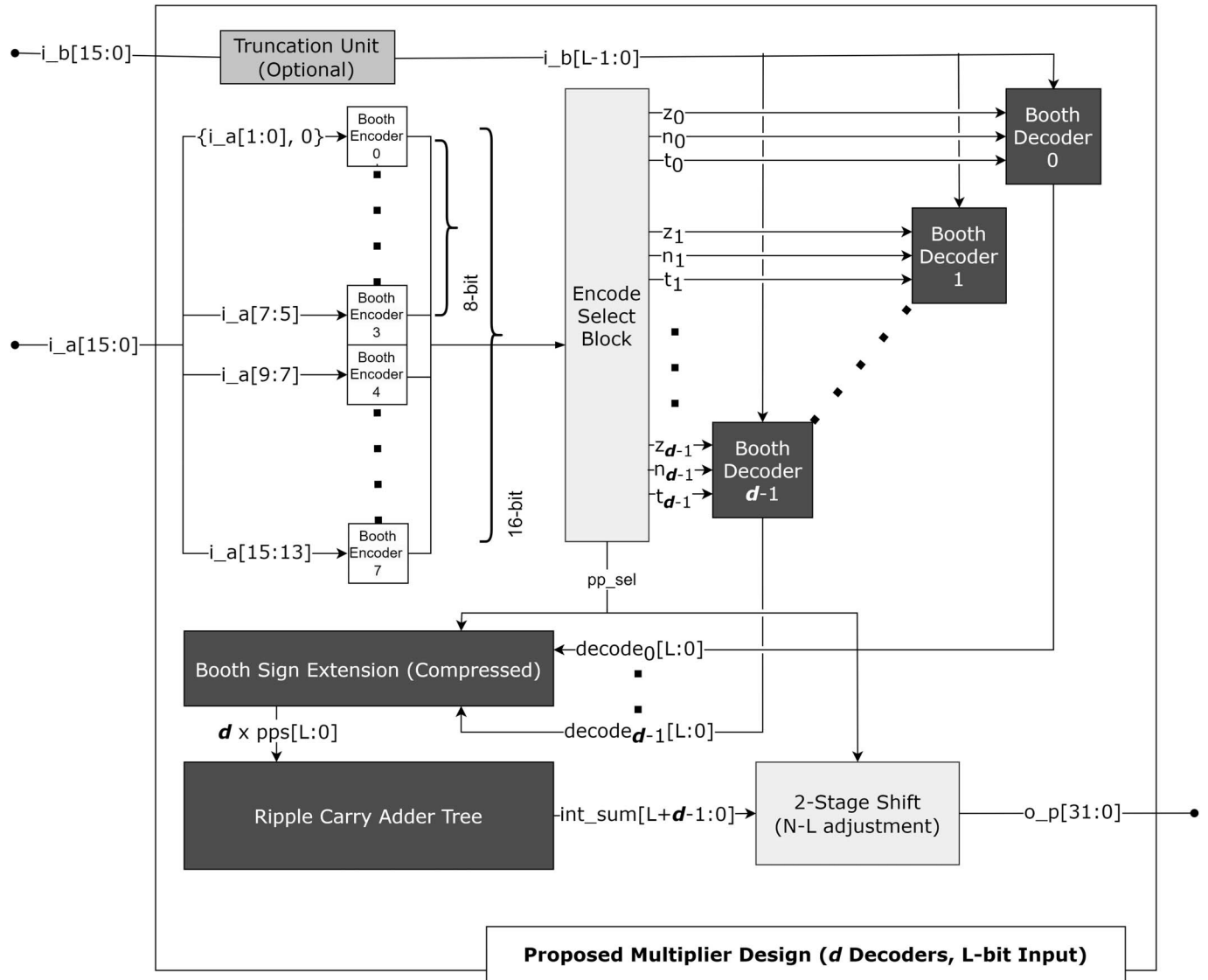


Fig. 2. The hardware architecture of the proposed approximate Booth multiplier using the proposed DRA scheme. The figure shows a parameterized general design that has all the possible modules included. The number of Booth encoder blocks depends on the overall bit-width of the multiplier (N). The diagram shows that in an 8-bit design, only Booth encoders from 0 to 3 would exist, whereas, all 8 Booth encoders would be needed for 16-bit designs. The number of Booth decoders d depends on the choice of the designer, however, d must always be less than $N/2$. Similarly, in designs where $d = 1$, there will be no Ripple Carry Adder Tree, since there is only decode. Finally, the design's second input can be truncated to L -bits to further reduce the computation complexity of the circuit and reduce resource consumption. (See Section II).

to the rest of the proposed 8-bit approximate multipliers. The compressed sign extension unit here takes in two of the decodes for appropriate shifting as dictated by the pp_sel signal. The adder tree ends up with an irregular shape due to an odd number of partial product rows, however, the proposed BD8.3 multiplier provides sizable gains in resource efficiency at a very insignificant error rate. Due to this, we enable the truncation unit in the proposed BD8.3 multiplier with the same options as the proposed BD8.2 multiplier. The truncation unit sets L to 8, 6, or 4.

B. Proposed 16-Bit ABMs Using the Proposed DRA Scheme

The effectiveness of the proposed DRA scheme is more obvious in the larger bit-width Booth multipliers. When the

proposed DRA scheme is applied to a 16-bit Booth multiplier, it can take 7 possible values for the W metric. Each of these values and their relative complexities are listed in Table I. Not all the variations are plausible for implementation in the hardware, since the logical circuits for selection of a set of encode out of 28, 56, or 70 possible choices would increase the critical path delay to an unreasonable value. Therefore, we develop and propose multiple grouping methods for the proposed BD16.4 multiplier for circuit simplification. The three proposed grouping methods are referred to as V1, V2, and V3. In this paper whenever BD16.4 is mentioned without the mention of V1, V2, or V3, it should be assumed that we are referring to BD16.4 V1. Additionally, we also implement the proposed BD16.1 and BD16.7 multipliers for experimentation as validation of the proposed DRA scheme.

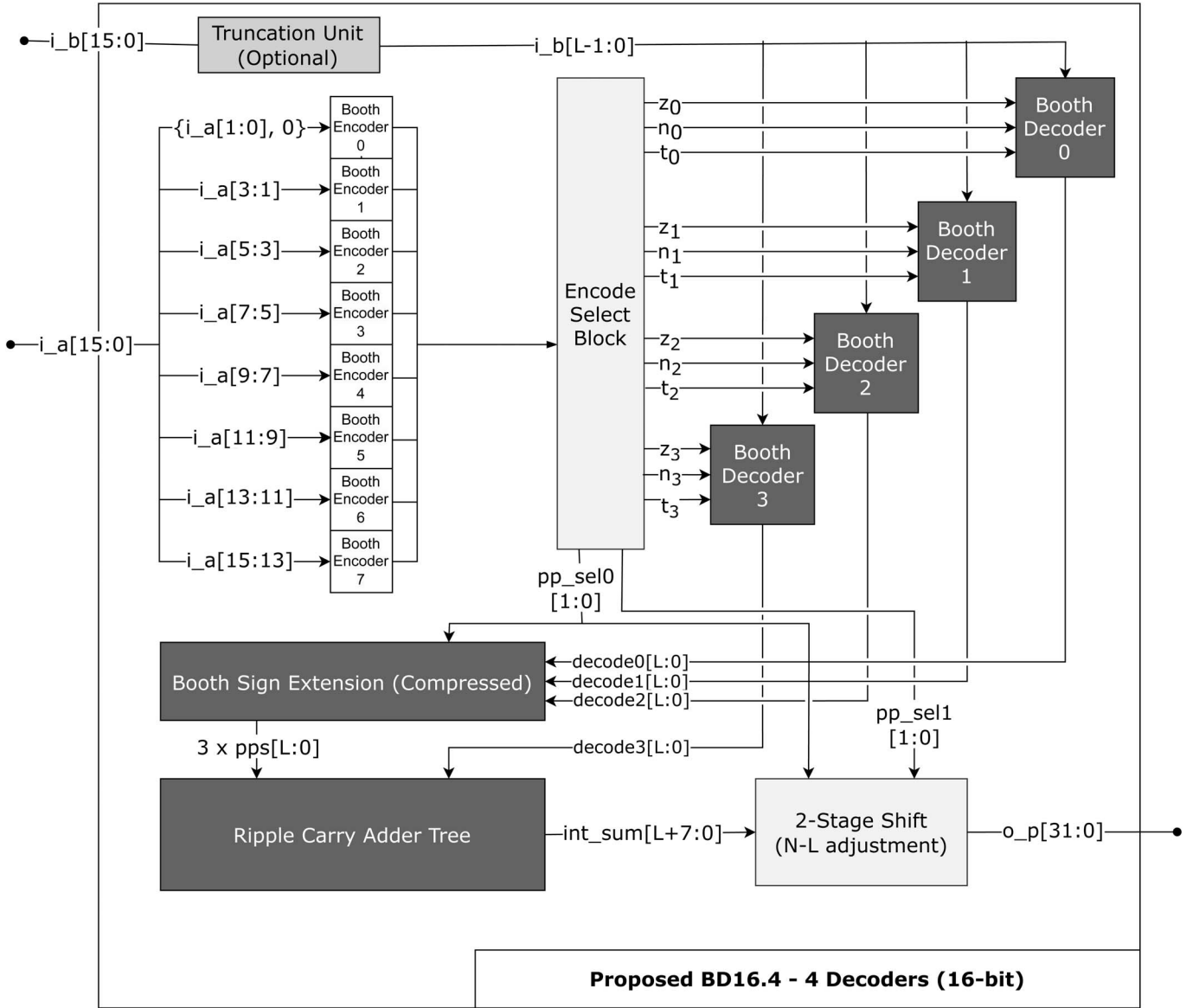


Fig. 3. The hardware architecture of the proposed BD16.4 multiplier, that is the proposed 16-bit approximate Booth multiplier with $W = 4$ decoders being used. W denotes the number of decoders specified under the proposed DRA scheme (See Section II).

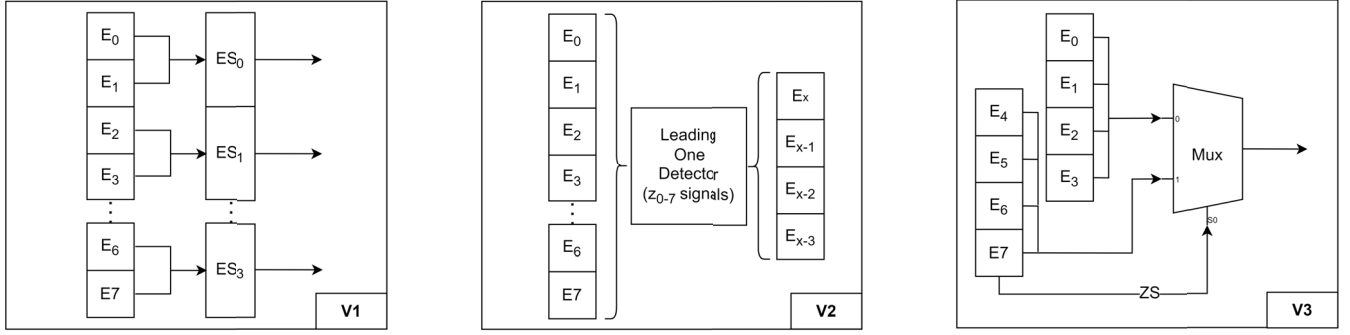
1) *BD16.1—Proposed Multiplier With $W = 1$* : For the implementation of the proposed BD16.1 multiplier, we start with the same architecture as the BD8.1. We extend all the internal units from 8-bit to 16-bit. Similarly, the shift unit is also updated to generate a 32-bit final output instead of the 16-bit output in the proposed BD8.1. This implementation shows similar characteristics as the proposed BD8.1, in that it has very high resource efficiency due to the generation of a single decode. However, due to such a high amount of reduction of computational elements, the resultant proposed BD16.1 multiplier is expected to have the highest error rate among all the proposed 16-bit ABMs.

2) *BD16.2—Proposed Multiplier With $W = 2$* : This variation holds no features of attraction. With an error rate slightly lower than the proposed BD16.1, and a significantly higher

implementation complexity (as shown in Table I). This variation is not implemented in the hardware.

3) *BD16.4 V1—Proposed Multiplier With $W = 4$* : All three versions of the proposed BD16.4 share the same overall architecture, as shown in Fig. 3. The architecture shown in the figure is an extension of the proposed BD8.3 multiplier architecture. It includes an additional Booth decoder, bringing the total amount up to 4. Similarly, the Booth sign extension unit is extended to output 3 partial products instead of 2. The adder tree is also extended to now add 4 partial products. Overall, the biggest change occurs in the *Encode Select Block (ESB)*. Here the multiplier can take up one of the three proposed grouping methodologies, as shown in Fig. 4.

The overall working principle of the proposed BD16.4 V1 is that it should remain as true to the proposed DRA scheme as



(a) A block diagram showing the most effective grouping technique for the proposed DRA scheme when the total possible choices get unreasonably large (See Table I). This method is most compatible with the proposed DRA scheme.

(b) A block diagram showing the leading-one-detection grouping scheme in practice for the reduction of total number of choices in 16-bit Booth multipliers (See Table I). This method is inspired by approximate logarithmic multipliers.

(c) A block diagram showing the simplest grouping scheme in practice for the reduction of a total number of choices in 16-bit Booth multipliers (See Table I). This method is included as a baseline for the analysis of test results.

Fig. 4. The hardware architecture of the three proposed grouping methods included in the proposed BD16.4 multiplier. See Section IV for further discussion.

possible. To achieve this effect, we make groups of two for each of the 8 incoming encodes in the ESB. The encode grouping is done using the following equation:

$$ES_x \leftarrow \{E_{2x+1}, E_{2x}\}; \quad 3 \geq x \geq 0 \quad (1)$$

This grouping results in the compression of incoming encodes into 4 groups instead of 8. These four groups shown as ES_x in Fig. 4, are then filtered using the same method as done in BD8.2 as shown in Table II in column $W = 2$, since now we need to choose 2 groups out of 4. The two groups generated are then expanded into four encodes which are fed into the four Booth decoders.

The proposed V1 grouping method is the most flexible compared to the V2 and V3 approaches and it closely follows the theory behind the proposed DRA scheme. Under this grouping method, the proposed BD16.4 has the freedom to choose between two disjointed groups of encodes. We show in Section IV that the proposed BD16.4 V1 outperforms the other two versions.

4) *BD16.4 V2—Proposed Multiplier With $W = 4$* : The proposed BD16.4 V2 employs a LOD circuit on all the incoming z_x (zero signals) from each of the encodes. The LOD circuit outputs the bit position of the leftmost non-zero encode which is then used in a function to generate the four chosen encodes. Given that x is the bit position of the leftmost non-zero encode, the generated four encodes are:

$$OutputEncodes(x) = \begin{cases} \{E_x, E_{x-1}, E_{x-2}, E_{x-3}\} & x \geq 4 \\ \{E_3, E_2, E_1, E_0\} & x < 4 \end{cases}$$

The proposed BD16.4 V2 adapts the theory behind the LOD-based approximation popularized in the approximate logarithmic multipliers. We adopt the LOD approximation scheme to work with our encode selection block in the proposed BD16.4 V2 multiplier. The rest of the components in the multiplier operate in the same fashion as in the proposed BD16.4 V1.

5) *BD16.4 V3—Proposed Multiplier With $W = 4$* : The proposed BD16.4 V3 employs a simple logic for the selection of encodes. A ZS signal is generated by applying an OR operation on z_{7-4} (zero signals for E_{7-4}). The ZS signal is 0 if E_{7-4} are all zero and it is 1 otherwise.

Depending on the value of the ZS the MUX either forwards the top half of the encodes or the bottom half. This methodology has the highest error rate among the three grouping methods discussed in this paper. We discuss the implications of each of the methods further in Section IV in the context of experimentation and hardware implementation.

6) *BD16.5 & BD16.6—Proposed Multipliers With $W = 5$ and $W = 6$* : These two variations are ignored due to the impressive results generated by the proposed BD16.4. The proposed BD16.5 and BD16.6 are expected to have an error rate between the proposed BD16.7 and BD16.4. The extremely low error rate of BD16.4 discourages the need for BD16.5 and BD16.6, as such they were not implemented in the hardware.

7) *BD16.7—Proposed Multiplier With $W = 7$* : The proposed BD16.7 multiplier is implemented to show that as the bit-width of the multiplier increases, the proposed DRA scheme causes an impactful reduction in the total number of decoders (8 to 7) and consequently a reduction in the rows of the partial product. Ultimately, we get enough simplification in the adder trees of the Booth multiplier to significantly improve the resource efficiency of the proposed BD16.7 multiplier as shown in Table IV. The proposed BD16.7 has a partial product array with an odd number of rows causing an irregular arrangement of adders in the accumulation tree, however, it still manages to beat the exact Booth multiplier in resource efficiency with a negligible error rate.

IV. EXPERIMENTS AND DISCUSSION

All the mentioned multipliers were implemented in TSMC 65-nm technology and analyzed using the Synopsys Design CompilerTM. The power calculations were made using the Synopsys Power AnalyzerTM. We set the transition rate to

TABLE III
THE IMPLEMENTATIONAL AND EXPERIMENTAL RESULTS FOR EACH OF THE 8-BIT MULTIPLIERS IN OUR EXPERIMENTS

Multiplier	Area(μm^2)	Delay(ns)	Power(μW)	PAP	PDP(fJ)	NMED(10^{-3})	MRED(%)	NN ^a Acc(%)
R4_BMULT8 ^b	823.30	1.03	3.1056	2556.84	4.111866	0	0	95.8
BD8.1*	155.9	0.52	1.5419	240.3822	0.801788	25	25.45	33.3
BD8.2* (L=4)	221.40	0.79	1.5406	341.0888	1.217074	13.95	35.6	57.7
BD8.2* (L=6)	286.92	0.80	1.6944	486.1572	1.35552	5.98	9.7	92.5
BD8.2* (L=8)	334.80	0.98	2.5029	837.9709	2.452842	5	4.8	93.7
BD8.3* (L=4)	340.56	0.88	2.4409	831.2729	2.147992	13.99	33.8	53.7
BD8.3* (L=6)	430.30	0.95	2.1339	918.2172	2.027205	3.5	8.1	95.8
BD8.3* (L=8)	514.30	1.03	2.5394	1306.013	2.615582	1.3	2.2	95.8
ABM-M1 [18]	375.48	0.87	3.1706	1190.50	2.758422	3.53	54.53	85.75
TL8_4_3 [12]	369.00	0.83	1.7897	660.3993	1.485451	23	-	14.25
DRALM8_4 [13]	443.30	0.70	1.8358	813.8101	1.28506	9.2	8.59	68.75
M_T6_C2[15]	529.50	1.18	1.8223	964.9079	2.150314	4.5	5	70.25

* The approximate multipliers proposed in this paper.

^a A 3-layer Linear Neural Network trained and tested on the MNIST handwritten digit dataset.

^b An exact 8-bit Booth multiplier for baseline.

TABLE IV
THE IMPLEMENTATIONAL AND EXPERIMENTAL RESULTS FOR EACH OF THE 16-BIT MULTIPLIERS IN OUR EXPERIMENTS

Multiplier	Area(μm^2)	Delay(ns)	Power(μW)	PAP	PDP(fJ)	NMED(10^{-3})	MRED(%)	NN ^a Acc(%)
R4_BMULT16 ^b	2460.60	1.84	11.2658	27720.63	20.729072	N/A	N/A	94.318
BD16.1*	442.4	0.9	3.8946	1722.971	3.50514	25.0393	24.31	5.53
BD16.4* (L=10)	928.08	1.61	4.7619	4419.424	7.666659	0.4999	1.62	7.22
BD16.4* (L=12)	1069.92	1.78	6.3709	6816.353	11.340202	0.4407	0.79	8.58
BD16.4* (L=14)	1211.40	1.85	7.7245	9357.459	14.290325	0.4346	0.59	85.227
BD16.4*	1302.10	1.88	7.952	10354.3	14.94976	0.4342	0.55	94.886
BD16.4* V2	1393.56	1.78	7.444	10373.66	13.25032	6.9192	74.22	68.789
BD16.4* V3	952.92	1.28	7.0915	6757.632	9.07712	8.76	85.55	59.443
BD16.7*	1873.80	1.75	10.003	18743.62	17.50525	0.0007	0.01603	94.118
TL 16-8/3 [12]	816.84	0.93	7.495	6122.216	6.97035	17.14	7.22	29.304
M T6 C1 ^c [15]	1665.72	1.18	6.934	11550.1	8.18212	16.89	5.9	71.12
M T8 C1 ^c [15]	1786.32	1.27	6.223	11116.27	7.90321	15.32	4.4	78.11
DRALM5 [13]	1423.44	0.98	9.93	14134.76	9.7314	9.23	6.88	69.71
ILMAE [14]	1485.36	1.87	10.986	16318.16	20.54382	10	9.51	61.89
RADI024 [17]	1680.48	1.28	10.1502	17057.21	12.992256	8.88	5.42	62.73
R4ABM1 [16]	2201.11	1.78	16.445	36197.25	29.2721	9.331	15	66.21
R4ABM2 [16]	2107.40	1.70	17.6045	37099.72	29.92765	6.234	21	70.01

* The approximate multipliers proposed in this paper.

^a A 3-layer Linear Neural Network trained and tested on the MNIST handwritten digit dataset.

^b An exact 16-bit Booth multiplier for baseline.

^c Abbreviation for MITCHEL_TRUNC_W6_C1 & MITCHEL_TRUNC_W8_C1 approximate multipliers from [15].

5%, the static probability to 20%, and the operating frequency to 10 MHz for all the designs considered in this experiment. Additionally, the multipliers taken from the literature were re-implemented and evaluated using the same conditions as the proposed designs to ensure a valid and fair comparison.

The experiments were conducted separately for 8-bit and 16-bit multipliers. The MNIST network is quantized to 8-bit integers and 16-bit integers using the TensorFlow LiteTM framework. The overall synthesis results and Neural Network (NN) accuracy results for the MNIST dataset are quoted in Table III for 8-bit experiments and in Table IV for 16-bit experiments. We highlight the significant versions of the proposed approximate Booth multipliers in each table, these highlighted multipliers are the most important among the proposed approximate multipliers. Additionally, the Normalized Mean Error Deviation (NMED) and Power-Delay-Product (PDP) trade-offs for each of the designs under consideration are visualized in Fig. 5 for

8-bit experiments and Fig. 6 for 16-bit experiments. In each of the charts, the multipliers that lie closer to the origin are relatively more attractive due to a lower error rate (NMED) and a lower PAP. In the results, we have provided both the PAP and PDP metrics for comparison. We include the PAP to highlight the combined reductions in area and power that occur in the multiplier hardware through the proposed DRA scheme and the proposed multipliers.

A. 8-Bit Implementation and Experiments

The proposed BD8.2 (L=6) multiplier is the most attractive option for minimum PAP and PDP with a reasonable NMED. The proposed BD8.2 (L=6) reduced the PAP by 81% compared to the exact 8-bit Booth multiplier and by 26.4% compared to the TL8_4_3. Additionally, the proposed BD8.2 (L=6) also reduced the NMED by 74% and PDP by 8.74% compared to

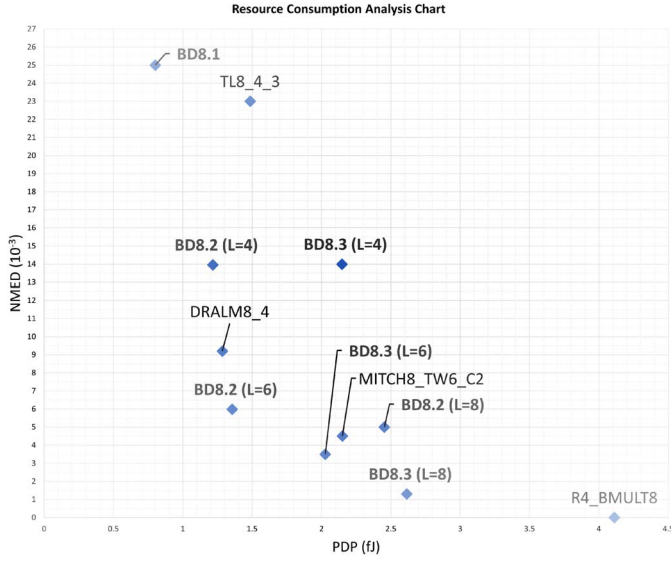


Fig. 5. A chart showing the trade-offs between NMED and PDP for different 8-bit multipliers included in our experiments. The chart is visualizing the data available in Table III. The proposed designs are shown with bold labels in the chart.

the TL8_4_3. Therefore, the proposed BD8.2 (L=6) multiplier operates at a significantly lower error rate compared to the state-of-the-art logarithmic multiplier which is effective in the DNN inference operation. The proposed BD8.2 (L=6) had 92.5% accuracy in the MNIST inference task compared to the accuracy of 14.25% of the state-of-the-art logarithmic multiplier. Consequently, the proposed BD8.2 (L=6) is the best candidate for those target platforms where a reasonable error tolerance is available but the multipliers are required to be extremely efficient.

Comparatively, the proposed BD8.3 (L=6) provides a significantly lower NMED compared to less resource-efficient approximate multipliers from the literature. The proposed BD8.3 (L=6) reduced the NMED by 62% compared to DRALM8_4 and by 23% compared to MITCH8_TW6_C2. Additionally, the proposed BD8.3 (L=6) also reduced the PAP by 5% and the PDP by 36.01% compared to MITCH8_TW6_C2. The proposed BD8.3 (L=6) reduces the NMED by 30% compared to the proposed BD8.2 (L=6) albeit with a higher PAP and PDP. We propose that the proposed BD8.3 (L=6) should be preferred for applications where a low error rate is highly encouraged.

Finally, the most accurate proposed multiplier is the proposed BD8.3 (L=8), with an exceptionally low NMED of 1.3 and an MRED(%) of 2.2%. The proposed BD8.3 (L=8) multiplier reduces the PAP by 48% and PDP by 50.57% compared to the exact 8-bit Booth multiplier. The proposed BD8.3 (L=8) multiplier showcases the significant resource efficiency possible with the proposed DRA scheme. The results agree with the initial theory presented in the previous sections. Additionally, the proposed BD8.2 (L=8) multiplier further increases the resource efficiency due to a higher simplification in the circuit layout. The proposed BD8.2 (L=8) reduces the PAP by 67% compared to an exact 8-bit Booth multiplier with an NMED of 5 and an MRED(%) of 4.8. The error rate is significantly

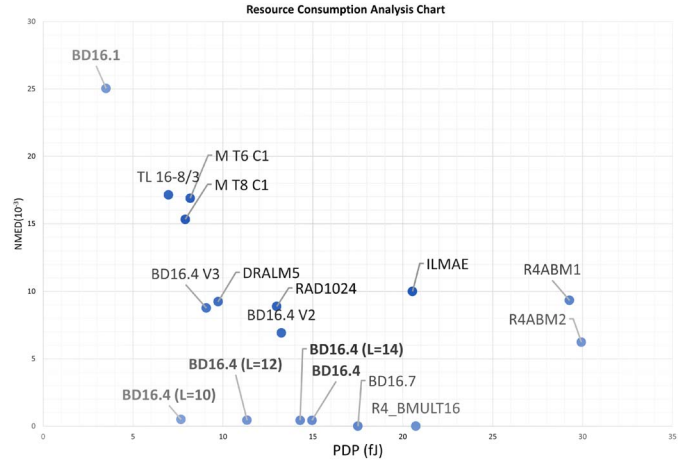


Fig. 6. A chart showing the trade-offs between NMED and PDP for different 16-bit multipliers included in our experiments. The chart is visualizing the data available in Table IV. The proposed designs are shown with bold labels in the chart.

less than the approximate multipliers from the literature. The proposed BD8.2 (L=8) multiplier contests very closely with the proposed BD8.3 (L=6) regarding resource efficiency. We prefer the proposed BD8.3 (L=6) due to its lower circuit delay for the applications that require real-time operation, however, the difference is negligible compared to the proposed BD8.2 (L=8).

The truncation methodology in our multipliers results in a higher relative error for smaller input values, therefore, in applications such as regression or digital signal processing where quantization to integer yields larger 16-bit integer values, we prefer BD8.2 (L=8). On the contrary, the higher resolution of BD8.3 (L=6) as a result of an additional decoder block results in a higher NN accuracy as the weights generally tend to normalize themselves around the zero value. The error rate in smaller values can often result in a sign flip for the final accumulated value, which corrupts the network operations as values might be blocked or passed at the ReLU layer. These errors then travel throughout the network ultimately reducing the overall accuracy.

The results quoted in this paper are further validated by testing all the designs with an NN inference task. The proposed BD8.2 (L=6, L=8) and the proposed BD8.3 (L=6, L=8) maintained excellent accuracy in the inference task with a negligible drop compared to the exact 8-bit Booth multiplier. All the significant designs proposed in this paperwork as expected in real-world conditions. They provide significant gains in resource efficiency in terms of power reduction and overall circuit area reduction compared to the state-of-the-art modern approximate multipliers and the exact 8-bit Booth multipliers.

B. 16-Bit Implementation and Experiments

Observing the Fig. 6, we note that six of the proposed 16-bit approximate multipliers are clustered relatively closer to the origin compared to the state-of-the-art approximate multipliers from the literature. Additionally, among those six the

proposed BD16.4 V2 and the proposed BD16.4 V3 are outperformed by the proposed BD16.4 and by the proposed BD16.4 (L=12). This highlights the advantage of the proposed DRA scheme coupled with the truncation, as it enables the multipliers to maintain a negligible error rate while significantly improving the overall resource efficiency of the multipliers.

Unlike the case with the 8-bit experiments where multiple proposed designs stood out among the competition, the 16-bit experiments show that the proposed BD16.4 (L=10) is the best implementation among all the designs considered. The proposed BD16.4 (L=10) can outperform the state-of-the-art approximate multipliers and the proposed BD16.7 design in terms of resource efficiency with a minimal NMED of 0.49 and the MRED(%) of 1.62. Such an error rate is too small to have any adverse impact on the application utilizing the proposed BD16.4 (L=10) multiplier. Overall, we set out to ask the question of whether the Booth multipliers need N/2 decoders, and we successfully demonstrate that the proposed BD16.4 (L=10) can work with N/4 decoders with minimal error rate using the proposed DRA scheme.

We note that the proposed BD16.4 multiplier reduces the NMED by 93% compared to the LOD-based proposed BD16.4 V2 multiplier with a similar PAP and a reduction in PDP by 42.14% as shown in Fig 6. If we draw a line from BD16.4 V2 to BD 16.4, the vertically downward movement reflects the reduction in NMED with a negligible increase in the PAP. When truncation is applied to the BD16.4 multiplier, we note that the point moves horizontally to BD16.4 (L=10). The horizontal movement towards the origin reflects that the PAP is significantly reduced with no increase in the NMED of the multiplier design. The resource analysis chart clearly shows that the proposed DRA scheme paired with truncation on one of the input ports can improve resource efficiency with a minimal error rate.

The proposed BD16.4 (L=10) and (L=12) truncate a significant amount of bits from the least significant side. Our BD16.4 V2 and BD16.4 V3 multipliers work well when the two inputs are either too large or too small. As such, in this specific experiment, since the weights of the NN tend to be normalized around the zero value, these multipliers can show a higher-than-expected performance in NN. However, these multipliers cannot hold their own under balanced validation testing which provides a full range of inputs as was used for the calculation of NMED and MRED of each multiplier in our experiments.

The proposed BD16.1 and the proposed BD16.7 take up two extremes in the resource chart shown in Fig. 6. The proposed BD16.7 has the highest PAP with the minimal NMED, while the proposed BD16.1 has the minimal PAP with the highest NMED. Neither of these designs showcases an attractive trade-off which can be verified by their large distance from the origin on the resource analysis chart. Comparatively, the BD16.4 and its truncation variations present a much better trade-off in terms of NMED vs PAP.

The proposed designs outperform all the state-of-the-art multipliers. The proposed BD16.4 (L=10) multiplier reduces the NMED by 92%, PAP by 88%, and PDP by 74% when compared to the approximate partial product generation paired with

TABLE V
THE IMPLEMENTATIONAL AND EXPERIMENTAL RESULTS FOR EACH OF THE 16-BIT MULTIPLIERS IN OUR EXPERIMENTS

Multiplier	Area (μm^2)	Delay (ns)	Power (μW)	Energy ^a (J)	Acc%
R4_BMULT8	823.30	1.03	3.1056	137.960	83.5
BD8.3 (L=8)	514.30	1.03	2.5394	112.808	71.93
BD8.3 (L=6)	430.30	0.95	2.1339	94.794	19.39
BD8.2 (L=6)	286.92	0.80	1.6944	75.270	24.5
BD8.2 (L=8)	334.80	0.98	2.5029	111.187	42.4
TL8_4_3	369.00	0.83	1.7897	79.504	23.45
DRALM8_4	443.30	0.70	1.8358	81.552	39.28
R4_BMULT16	2460.60	1.84	11.2658	500.462	83.5
BD16.4 (L=10)	928.08	1.61	4.7619	211.538	21.7
BD16.4 (L=14)	1211.40	1.85	7.7245	343.146	55.8
BD16.4	1302.10	1.88	7.952	353.252	81.3
TL 16-8/3	816.84	0.93	7.495	332.951	54.2
M T6 C1	1665.72	1.18	6.934	308.030	49.2
DRALM5	1423.44	0.98	9.93	441.121	46.7

^a Energy is calculated using the total time for inference by ResNet-50 per sample in the test set from the CIFAR-10 dataset. The total run-time is multiplied by the power of the multiplier as derived by Synopsys PowerCompiler.

the truncation method (R4ABM1 & R4ABM2). The proposed BD16.4 (L=10) also reduces the NMED by 96.5%, PAP by 69.6%, and PDP by 21.22% when compared to the state-of-the-art logarithmic multiplier (DRALM5—best performing approximate logarithmic multiplier in our experiments). The proposed BD16.4 (L=10) also reduces the NMED by 94.4%, PAP by 74.8%, and PDP by 41% when compared to the non-logarithmic RAD1024 higher-radix approximate Booth multiplier.

C. ResNet-50 Experiments

We trained the ResNet-50 network on the CIFAR-10 dataset achieving an 82.3% testing accuracy using an exact multiplier. We chose the best-performing multipliers from Table IV for further testing on complex neural networks to demonstrate the viability of the proposed ABMs. The results of the ResNet-50 experiments are listed in Table V. We quantized the default ResNet-50 model available in the TensorFlow library using the TFLite framework in 8-bit integer format. All multiplication operations in the network are then offloaded to the approximate multipliers under test, and the overall network accuracy is measured. The proposed BD16.4 multiplier shows the smallest decrease in accuracy. However, the overall energy consumption of the proposed multipliers is twice that of the state-of-the-art logarithmic multipliers. We highlight that the proposed work has significantly improved the state-of-the-art ABMs. The drop in accuracy for the proposed BD16.4 (L=14) and the proposed BD16.4 (L=10) is primarily due to the truncation applied. The weights in the ResNet-50 are normalized around the zero value and as such the truncation makes the multiplier significantly more error-prone in low-magnitude inputs like the weights in a quantized NN.

On the other hand, the proposed BD8.3 (L=8) outperforms the rest of the 8-bit approximate multipliers in this experiment. Our testbench implementation caused the 8-bit approximate

multipliers to be at a disadvantage since they had to perform 16-bit multiplications in 4 cycles compared to a single cycle computation by a 16-bit approximate multiplier. We will discuss the implications of this implementation in detail in the following section. Overall, the proposed BD8.3 (L=8) provides a significant reduction in the energy consumption for the ResNet inference at a significant loss in accuracy, however, this degradation is the smallest among all the other approximate multipliers.

We observed that avoiding the application of approximate multipliers in the final dense layers of the ResNet-50 model resulted in an average accuracy increase of about 10% across the board except for the proposed BD16.4 multiplier and the proposed BD8.3 (L=8) multiplier. We believe that this occurs due to the negligible error rate of these multipliers compared to the rest of the approximate multipliers. The final layers of complex networks are expected to be sensitive to errors introduced as the ReLU activation is not available (in the last layer) and the errors have a higher impact compared to the earlier convolutional blocks where the residual path/connection can offset some of the errors introduced. However, we refrained from including such in the tables since the networks are fully deployed on the approximate multiplier under test in this experiment.

The proposed BD16.4 multiplier has shown the best accuracy across all the multipliers. One of the major evidence for the deployment of proposed approximate networks is their significant improvement in reliability over the existing state-of-the-art ABMs. Approximate computing is generally avoided in critical DNN applications, where a threshold accuracy must be met, such as obstacle detection in autonomous cars or medical image diagnosis in healthcare applications. Through the proposed work we demonstrate and promote further research into reliable approximate computing systems for potential deployment in such environments for inference of critical and complex DNN networks enabling AI inference at the edge.

D. Final Thoughts

The proposed DRA scheme and the proposed ABMs offer a variety of error patterns, each suitable to a potentially unique application. There are two approximation criteria at play in the proposed DRA scheme: decoder count and truncation amount.

1) *Decoder Count*: The proposed multipliers with a higher decoder count (such as the proposed BD16.4 and the proposed BD8.3) should be preferred in applications where the input has an even spread over the full range of the possible inputs. In applications such as general-purpose processors or generic vector accelerators, having the additional flexibility of handling a wider range of inputs is critical.

2) *Truncation Amount*: The truncation amount is a feature for fine-tuning the proposed ABMs to judge the limit of error tolerance in the proposed designs. We included this in our experiments to judge the robustness of the proposed DRA scheme under extreme conditions. The inputs should only be truncated if the provided error rate is safely above the minimum requirement of the application and hardware resources are expected to be extremely scarce.

3) *ResNet Testbench Analysis*: The NN inference testing performed in [12] is through using a fixed-point network architecture implemented in Caffe using C++. There are several differences here with our implementation. Since the multipliers are using fixed point weights and activations, the multiplication is performed once per value and then the result is generated by shifting the product. This arithmetic process improves the accuracy of the NN as the impact of approximation is reduced (only one multiplication per input pair). But this method is unfeasible for resource-constrained edge micro-controllers which often only use integer-only systems. Instead, we implement our NNs using TFLite int8 quantization. It is a comprehensive experimental setup compared to only using the approximate multipliers in the convolutional and dense layers of the networks [12]. The weights and inputs lie within the range of $[-127, 127]$ along with an offset value that can range from $+128$ to 0 . When added together, the resultant can exceed the signed 8-bit format. Therefore, the result must be multiplied in 16-bit format. Here, the 16-bit multiplication is performed in 4 steps by 8-bit multipliers (thus, the error rate is increased by $\times 4$). Since the 16-bit inputs to the multipliers often lie within the range of 9 bits (slightly out of the range of signed 8-bit integers), they are small and rely heavily on the first 2-6 LSB. Therefore, the truncation impacts the overall network accuracy more severely in 16-bit multipliers. It is in this situation, integer-only arithmetic, that the proposed DRA multipliers outperform the ALMs. Additionally, we see similar truncation causing severe degradation problems in the proposed BD16.4 (L=12) and the proposed BD16.4 (L=10).

V. CONCLUSION

In this paper, we have introduced a novel approach called the Decoder Reduction Approximation (DRA) scheme for implementing approximate Booth multipliers. By reducing the total number of Booth decoders, our proposed designs achieve better resource efficiency with minimal error rates compared to existing state-of-the-art approximate multipliers. Through the proposed DRA scheme, an approximate Booth multiplier with a bit-width of N can be implemented with W Booth decoders, such that:

$$W < D_{max}$$

$$D_{max} = N/2$$

where W is the specified number of Booth decoders under the proposed DRA scheme and D_{max} is the maximum number of Booth decoders possible in a traditional Booth multiplier.

Our experiments show that the proposed BD8.2 (L=10) and BD8.2 (L=6) 8-bit multipliers outperform the existing state-of-the-art logarithmic multipliers in terms of PAP, PDP, and NMED. The proposed BD8.2 (L=10) reduced the (PAP, PDP) by (81%, 67%) compared to the exact 8-bit Booth multiplier and by (26.4%, 8.74%) compared to the TL8_4_3 [12]. Additionally, the proposed BD8.2 (L=6) also reduced the NMED by 74% compared to the TL8_4_3. Therefore, the proposed BD8.2 (L=6) multiplier operates at a significantly lower error rate compared to the state-of-the-art logarithmic multiplier which

is effective in the DNN inference operation. The proposed BD8.2 (L=6) had 92.5% accuracy in the MNIST inference task compared to the accuracy of 14.25% of the state-of-the-art logarithmic multiplier.

Similarly, in the 16-bit experiments, we successfully demonstrated that the proposed BD16.4 (L=10) can outperform the proposed BD16.4 V2 (LOD-based) multiplier by reducing the NMED by 93% without any increase in the PAP and a decrease in PDP of 42.14%. The proposed BD16.4 also outperforms all of the existing state-of-the-art logarithmic multipliers by reducing the NMED by up to 96.5%, PAP by up to 88%, and PDP by up to 62.68%. We have successfully demonstrated that through the proposed DRA scheme, the proposed approximate Booth multipliers are successful in modern DNN applications such as ResNet-50 inference on the CIFAR-10 dataset.

We have successfully demonstrated that the proposed DRA scheme works well with larger bit-widths in Booth multipliers. The considerable resource efficiency of the proposed approximate Booth multipliers over the existing state-of-the-art works makes them well-suited to efficient error-tolerant applications such as AI inference on edge devices and low-power digital signal processing.

ACKNOWLEDGMENT

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC).

REFERENCES

- [1] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Survey and benchmarking of machine learning accelerators," in *Proc. IEEE High Perform. Extreme Comput. Conf. (HPEC)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1–9.
- [2] T. Fritzmam, G. Sigl, and J. Sepúlveda, "RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2020, no. 4, 2020, pp. 239–280.
- [3] M. Asadikouhanjani and S.-B. Ko, "Enhancing the utilization of processing elements in spatial deep neural network accelerators," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 9, pp. 1947–1951, Sep. 2021.
- [4] M. Asadikouhanjani, H. Zhang, L. Gopalakrishnan, H.-J. Lee, and S.-B. Ko, "A real-time architecture for pruning the effectual computations in deep neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 2030–2041, May 2021.
- [5] Y. Dou, C. Wang, R. Woods, and W. Liu, "ENAP: An efficient number-aware pruning framework for design space exploration of approximate configurations," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 2062–2073, May 2023.
- [6] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proc. IEEE*, vol. 108, no. 12, pp. 2108–2135, Dec. 2020.
- [7] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi, and F. Lombardi, "Design and evaluation of approximate logarithmic multipliers for low power error-tolerant applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 9, pp. 2856–2868, Sep. 2018.
- [8] S. Vahdat, M. Kamal, A. Afzali-Kusha, and M. Pedram, "TOSAM: An energy-efficient truncation- and rounding-based scalable approximate multiplier," *IEEE Trans. Very Large Scale Integr.*, vol. 27, no. 5, pp. 1161–1173, May 2019.
- [9] S. Vahdat, M. Kamal, A. Afzali-Kusha, and M. Pedram, "LETAM: A low energy truncation-based approximate multiplier," *Comput. Elect. Eng.*, vol. 63, no. C, pp. 1–17, Oct. 2017.
- [10] K. Abed and R. Siferd, "VLSI implementations of low-power leading-one detector circuits," in *Proc. IEEE SoutheastCon*, 2006, pp. 279–284.
- [11] A. Malik and S.-B. Ko, "Effective implementation of floating-point adder using pipelined LOP in FPGAs," in *Proc. Can. Conf. Elect. Comput. Eng.*, 2005, pp. 706–709.
- [12] R. Pilipović, P. Bulić, and U. Lotrič, "A two-stage operand trimming approximate logarithmic multiplier," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 6, pp. 2535–2545, Jun. 2021.
- [13] P. Yin, C. Wang, H. Waris, W. Liu, Y. Han, and F. Lombardi, "Design and analysis of energy-efficient dynamic range approximate logarithmic multipliers for machine learning," *IEEE Trans. Sustain. Comput.*, vol. 6, no. 4, pp. 612–625, Oct./Dec. 2021.
- [14] M. S. Ansari, B. F. Cockburn, and J. Han, "An improved logarithmic multiplier for energy-efficient neural computing," *IEEE Trans. Comput.*, vol. 70, no. 4, pp. 614–625, Apr. 2021.
- [15] M. S. Kim, A. A. D. Barrio, L. T. Oliveira, R. Hermida, and N. Bagherzadeh, "Efficient Mitchell's approximate log multipliers for convolutional neural networks," *IEEE Trans. Comput.*, vol. 68, no. 5, pp. 660–675, May 2019.
- [16] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, and F. Lombardi, "Design of approximate radix-4 booth multipliers for error-tolerant computing," *IEEE Trans. Comput.*, vol. 66, no. 8, pp. 1435–1441, Aug. 2017.
- [17] V. Leon, G. Zervakis, D. Soudris, and K. Pekmestzi, "Approximate hybrid high radix encoding for energy-efficient inexact multipliers," *IEEE Trans. Very Large Scale Integr.*, vol. 26, no. 3, pp. 421–430, Mar. 2018.
- [18] S. Venkatachalam, E. Adams, H. J. Lee, and S.-B. Ko, "Design and analysis of area and power efficient approximate booth multipliers," *IEEE Trans. Comput.*, vol. 68, no. 11, pp. 1697–1703, Nov. 2019.
- [19] S.-R. Kuang, J.-P. Wang, and C.-Y. Guo, "Modified booth multipliers with a regular partial product array," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 56, no. 5, pp. 404–408, May 2009.
- [20] M. H. Haider and S.-B. Ko, "Booth encoding based energy efficient multipliers for deep learning systems," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 70, no. 6, pp. 2241–2245, Jun. 2023.



Muhammad Hamis Haider received the B.E. degree in electrical engineering from the National University of Sciences and Technology, Islamabad, Pakistan. He is currently working toward the Ph.D. degree with the Department of Electrical and Computer Engineering, University of Saskatchewan, SK, Canada. His research interests include computer architecture, reconfigurable computing, and accelerator design for biomedical applications.



Hao Zhang (Member, IEEE) received the Ph.D. degree in electrical and computing engineering from the University of Saskatchewan, SK, Canada, in 2019. He worked as a Postdoctoral Fellow with the Department of Electrical and Computer Engineering, University of Saskatchewan, from 2019 to 2020. He is currently an Associate Professor with the Faculty of Information Science and Engineering, Ocean University of China, Qingdao, China. His research interests include computer arithmetic, computer architecture, reconfigurable computing, and hardware accelerators for deep learning. He is a member of the IEEE Computer Society and IEEE Circuits and Systems Society.



Seok-Bum Ko (Senior Member, IEEE) received the Ph.D. degree from the University of Rhode Island, USA, in 2002. He is currently a Professor with the Department of Electrical and Computer Engineering and the Division of Biomedical Engineering, University of Saskatchewan, Canada. His areas of research interests include computer architecture/arithmetic, efficient hardware implementation of compute-intensive applications, deep learning processor architecture, and biomedical engineering. He is a Senior Member of the IEEE Circuits and Systems Society and an Associate Editor for the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) and the IEEE ACCESS. He is also an Academic Editor for IET Computers & Digital Techniques. He was an Associate Editor for the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS (2019–2021).